

Reviewer Pack — Minimal Index of Poincaré Duality and Communication Complexi...

Entry 4b56c2ce3d5c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 19:14:35 UTC

Minimal Index of Poincaré Duality and Communication Complexity Rank

Entry ID: 4b56c2ce3d5c **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 19:14:35 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Poincaré duality) **Field B** (complexity object): Communication Complexity

Statement:

The minimal index of Poincaré dual lattices in a given problem instance is linearly correlated with its communication complexity rank, such that the index of Poincaré duality $I(G) = \Theta(r(\varphi_G))$ for the associated formula φ_G .

Rationale (proposer's reasoning):

Poincaré duality relates to the topological structure of spaces and might provide a new perspective on the organization of information in communication complexity. This conjecture suggests that the algebraic topology of problem instances could be a novel source of insights into their computational hardness.

Taxonomy category: Poincare_Duality (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): bf0070932e417b75

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between $I(G)$ and $r(\varphi_G)$ exceeds 0.8 across all instances, and no instance has an $I(G) > 2 * r(\varphi_G)$. It is falsified if any seed yields a Pearson correlation coefficient < 0.5 or an $I(G) > 1.5 * r(\varphi_G)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Poincaré duality" AND "communication complexity" AND lattice - "minimal index" IN ALGEBRAIC TOPOLOGY AND communication complexity rank - "Poincaré dual lattices" related to communication complexity

Top relevant hits considered: - [http://arxiv.org/abs/math/0502195v2] Hopf algebra structure on topological Hochschild homology - [http://arxiv.org/abs/1612.03133v3] Topological Complexity of the Klein bottle - [http://arxiv.org/abs/1504.04733v2] On the geometry and topology of partial configuration spaces of Riemann surfaces - [http://arxiv.org/abs/2401.14623v1] Structure in Communication Complexity and Constant-Cost Complexity Classes - [http://arxiv.org/abs/1403.8106v1] Recent advances on the log-rank conjecture in communication complexity - [http://arxiv.org/abs/nlin/0101006v1] On Complexity and Emergence

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Define constants and parameters for testing
    n_tests = 30
    instances_tested = 0
    metric_values = []
    n_max = 5
    conjecture_holds = True
    counterexample = ""

    for _ in range(n_tests):
        # Generate a random instance  $\phi_G$  (simplified example)
        n = random.randint(5, 40)
        instances_tested += 1
        n_max = max(n_max, n)

        # Compute the Poincaré dual lattice index  $I(G)$  (simplified example)
        I_G = random.uniform(1, 2 * n)

        # Measure the communication complexity rank  $r(\phi_G)$  (simplified example)
        r_phi_G = random.uniform(0.5, n / 2)
```

```

# Store the metric value
metric_values.append(I_G)

# Check if the conjecture holds for this instance
if I_G > 1.5 * r_phi_G:
    conjecture_holds = False
    counterexample = f"Instance with  $I(G)=\{I_G\}$  and  $r(\phi_G)=\{r\_phi\_G\}$ "

# Compute the Pearson correlation coefficient (simplified example)
mean_I_G = sum(metric_values) / len(metric_values)
variance_I_G = sum((x - mean_I_G) ** 2 for x in metric_values) / len(metric_values)
covariance = sum((metric_values[i] - mean_I_G) * (r_phi_G - r_phi_G) for i, r_phi_G in enumerate(r_phi_Gs)) / len(r_phi_Gs)
std_r_phi_G = math.sqrt(sum((x - r_phi_G) ** 2 for x in r_phi_Gs) / len(r_phi_Gs))
correlation_coefficient = covariance / (std_I_G * std_r_phi_G)

# Check if the conjecture is supported
if correlation_coefficient < 0.5:
    conjecture_holds = False
    counterexample = f"Pearson correlation coefficient {correlation_coefficient} < 0.5"

return {
    "metric_name": "Pearson correlation coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys

    seeds = [int(x) for x in sys.argv[1:]] or [random.randint(2, 1000) for _ in range(30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={result['counterexample']} first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8b8ce220.py", line 77, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8b8ce220.py", line 52, in run_trial
    covariance = sum((metric_values[i] - mean_I_G) * (r_phi_G - r_phi_G) for i, r_phi_G in enumerate(r_
1)
```

^^^^^^^^^^^^^^^^^^^^

TypeError: 'float' object is not iterable

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the Pearson correlation coefficient and comparing it with the required thresholds. | next: Review the test code for potential bugs that cause it to crash and fix them. Once fixed, rerun the test to verify if the conjecture meets the pre-registered support conditions.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13695
2	propose	ollama_remote	glm4:latest	0	0	12970
3	preregistration	ollama_remote	glm4:latest	0	0	11905
4	novelty	ollama_remote	glm4:latest	0	0	11862
5	novelty	ollama_remote	glm4:latest	0	0	10624
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21691
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13576
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20237
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17247
10	judge	ollama_remote	glm4:latest	0	0	13865

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 147671 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in [research/audit/4b56c2ce3d5c.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4b56c2ce3d5c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/4b56c2ce3d5c.tar.gz` (if generated)